

DecisionBroker — Project Overview (Live)

Living document. Describes **what this project is trying to do and why** — the problem, the vision, the value, and the scope of the proof of concept. It is deliberately about **goals, not implementation**. For *how the code is organized* see `architecture.md` ; for *what it's built with* see `tech stack.md` .

Audience: layered on purpose. The **Vision & Value** sections speak to anyone (stakeholders, new contributors); **Scope of the Proof of Concept** onward is the working agreement for the author and the AI/agent sessions building it.

Last updated: 2026-06-13

1. In one sentence

DecisionBroker is a collaboration layer between AI sessions and the people who need to make the decisions those sessions get stuck on — it lets a question leave an AI session, be routed to the right human (or to a rule), get answered, and flow back into the session, so that one person's judgment can scale to a team and the decisions made along the way accumulate into reusable assets.

2. The problem (why this exists)

Working with AI is dramatically more effective when the AI drives the dialog — asking *you* for the next decision rather than you trying to specify everything up front. A decision-tree / "grill-me" style of prompting works well because it:

1. walks you through a logical sequence so unknowns get resolved systematically, and
2. keeps your understanding and the AI's understanding tightly aligned.

But that pattern has friction and missed potential:

- **Attention is scattered.** Because each AI turn can take 10–30 minutes, you run 3–5 sessions in parallel and spend your time flipping between terminals

hunting for "where was I, and what does this one need from me next?"

- **It doesn't scale past one person.** Every prompt today can only go to *you*. There's no way to direct a particular question to the teammate who should actually answer it, or to let a whole team participate through one interface.
- **Decisions evaporate.** The same or similar questions get asked over and over, and the answers — real organizational knowledge — are never captured as something durable that could answer the next instance automatically.
- **It requires expertise.** The whole loop lives in a terminal, behind Git and Claude and CLI fluency that most people in an organization don't have.

3. The opportunity

If questions can be *brokered* — routed away from the terminal to the right person and back — several things become possible at once:

- **Scale one person's judgment to a team.** Questions go to whoever should answer them; a group can collaborate through a single question interface instead of one expert being the bottleneck.
- **Turn decisions into corporate assets.** When a question recurs, capture a **rule** — a first-class, logged, maintained artifact — that supplies the right context, answers it automatically, or routes it onward. The accumulated rules become organizational knowledge and an operating manual.
- **Let workflows emerge organically.** Instead of designing a rigid workflow up front, each person optimizes their own corner (triage, enrich, route, invoke a skill). The *collection* of everyone's rules ends up *being* the company's workflow — bottom-up rather than top-down.
- **Reach non-technical users.** If you can use email, you can use DecisionBroker. No terminal, no Git, no Claude expertise required to participate.

4. The mental model

Think of it as **an inbox for decisions, shared between humans and AI sessions.**

- An AI session (or an app, email, chat message, etc.) **emits a request** — a question, an approval, a choice, or a "please add detail" ask.
- The request is **structured**, can carry **attachments** (designs, palettes, PDFs, images, any file), and is **routed** to an inbox.

- People **monitor inboxes** the way they monitor email — individual inboxes, shared group inboxes, and workflow inboxes.
- Opening a request shows a **dynamically generated interface** built from the request's contents (e.g. a form with links to its attachments).
- The responder can **answer, forward, or write a rule** — and a rule might itself answer, request consensus from a group, forward to someone else, or bounce the request back to its originator for missing information (creating a dialog).
- The answer **flows back** to whatever emitted the request — including back into a waiting AI session.

Working vocabulary

Term	Meaning
Work item / decision request	A single unit needing a human (or rule) response. "Decision request" can mislead — some items just ask for more content — so work item is the more general term.
Connector	How a work item enters or leaves the system (CLI/MCP, web, app, email, Slack, Telegram, ...). A generic in/out boundary.
Inbox	Where routed work items wait to be seen. Individual, group, or workflow.
Rule	A first-class, logged artifact that can auto-answer, enrich, route, request consensus, or bounce a work item. Rules are the captured-knowledge asset.
Routing	The information attached to a work item that says which inbox(es) it belongs to.

5. Goals and non-goals (the product, long-term)

Goals

- Decouple the "ask the user" moment from the terminal so a question can reach the right person and return.
- Make any work item a structured, attachment-carrying, routable object.
- Give non-technical users an email-grade way to receive and answer requests.
- Capture decisions as durable, reusable rules.

- Let workflows form organically out of individuals' rules.
- Support enterprise adoption where a company can run this **inside** its boundary and never share data outside the organization (WorkOS for identity).
- Spread organically — let an unauthenticated person participate via a shared URL, with sign-up/app encouraged but not required.

Non-goals (for now)

- Not a rigid, pre-designed BPM/workflow engine — workflows emerge from rules.
- Not a vendor-locked SaaS — large portions are intended to be **open source**, with an optional paid hosted tier; companies must never feel trapped.
- Not limited to software-development decisions — that's just the first dogfood.

6. Guiding principles

- **Open by default.** Open-source the core; monetize hosting/operation, not lock-in. (Consistent with the existing portability principle in `tech_stack.md`.)
- **Portability / no lock-in.** Standard Postgres, standard drivers, swappable adapters — already the house style; the same applies to identity (WorkOS, self-hostable) and storage.
- **Non-technical first.** The bar is "if you can use email, you can use it."
- **Standards over invention** for the work-item format, so one declarative description can render on web, iOS, and Android.
- **Data stays home.** Enterprises can run it within their own boundary.

7. Scope of the proof of concept

The PoC's job is **not** production quality. Its job is to **demonstrate that all the elements can work together** — to retire the central risk that the round-trip is real and useful — and to surface options and risks for the parts that are still fuzzy.

7.1 The spine: a thin end-to-end slice

Build the **walking skeleton** — narrow but complete — that touches every part of the loop once rather than building any one part deeply:

AI session emits a question → it becomes a structured work item → it lands in an inbox → a human routes/answers it → the answer flows back into the waiting AI session → and at least one **rule** can short-circuit a future instance.

Depth (rich forms, many connectors, a real rules engine, consensus, archival) comes *after* the skeleton walks.

7.2 The canonical demo (definition of "it works")

The concrete moment that validates the idea — drawn from the author's real pain:

1. A Claude session building a feature hits a **design fork** and, instead of prompting the terminal, emits the question to DecisionBroker.
2. The work item lands in **the author's own inbox** first.
3. The author **forwards** it to a **teammate's inbox** (exercising routing + forwarding, not just a direct hop).
4. The teammate **answers** through a dynamically rendered interface.
5. The answer **flows back into the original AI session**, which continues as if the user had typed it.

If that sequence works, the core thesis is proven.

7.3 Decisions locked for the PoC

Decision	Choice for the PoC	Rationale
How AI sessions tap in	An MCP server/tool . DecisionBroker exposes an ask -style tool the AI calls instead of its built-in prompt; the tool posts the work item and blocks until answered.	Standard, robust, no custom terminal to build. (The original "custom terminal that Claude runs inside" idea is deferred — see Open Questions.)
User scope	Two users with routing — author's inbox → forward → teammate's inbox.	Proves "scale myself to others," which single-user can't, without full group-inbox overhead.
Work-item format	Minimal custom JSON now , just enough for the demo.	Keep momentum; picking a real declarative-form standard is an explicit early research task, not a blocker.

Decision	Choice for the PoC	Rationale
Doc/asset surface	One rule that can auto-answer or auto-route a repeat of the demo question.	Demonstrates the "decisions become assets" value at skeleton depth.

7.4 Explicitly out of scope for the PoC

Email/Slack/Telegram connectors; production auth/permissions; mobile clients; unauthenticated shared-URL access; consensus/quorum answering; attachment archival lifecycle; a general rules *engine* (one or two hard-coded-ish rules is enough); polish. These are real goals — just **not** what the PoC must prove.

8. Open questions to resolve (knowingly unsettled)

- **Work-item format standard.** Which declarative-form standard renders across web/iOS/Android? (JSON Schema + a renderer, Adaptive Cards, something else.) The PoC uses minimal JSON; choosing the real standard is a near-term research task.
- **The blocking round-trip.** How does an AI session *wait* for an out-of-band human answer cleanly (timeouts, multiple parallel sessions, a session that ends before the answer arrives)?
- **Custom terminal vs MCP, long-term.** MCP is the PoC choice; is a custom terminal ever worth it for capturing prompts the AI *doesn't* route through a tool?
- **Rule model.** What is a rule, exactly — its trigger, its action vocabulary (answer / enrich / route / forward / request-consensus / bounce / invoke a skill), and how is it stored and versioned as a first-class artifact?
- **Routing & inbox model.** How are inboxes, membership, and routing metadata modeled? (Feeds the still-to-be-written `data-model.md`.)
- **Unauthenticated participation.** How can a shared URL let an outsider answer safely without an account?

9. Risks

Risk	Why it matters	How the PoC addresses it
The round-trip is awkward or slow — the core bet.	If routing a question out and back is clunkier than just answering in-terminal, the idea fails.	The thin slice exists precisely to feel this end-to-end early.
Blocking an AI session on a human is fragile.	Parallel sessions, long waits, dead sessions.	Prove the happy path first; document failure modes as findings.
Format lock-in.	Wrong work-item format is expensive to undo across three clients.	Minimal JSON now + a deliberate standards survey before committing.
Scope explosion.	The vision is enormous; easy to over-build.	Hard out-of-scope list; skeleton-first discipline.
Security/data-isolation later.	Enterprises won't adopt if data leaves their boundary.	Out of scope for the PoC, but kept portable (WorkOS, swappable adapters) so it isn't painted into a corner.

10. Recommended stages

De-risk by exploring and proving the riskiest, most novel part first; defer breadth.

- **Stage 0 — Spike the tap-in.** Stand up a minimal MCP `ask` tool that posts a hard-coded question somewhere and blocks for a typed answer that returns to the session. Proves the round-trip mechanism in isolation. (*Smallest possible test of the central bet.*)
- **Stage 1 — Work item + inbox.** Persist the question as a structured JSON work item (Postgres), with a bare web inbox that lists items and renders a minimal form to answer one. Answer flows back via Stage 0's channel.
- **Stage 2 — Routing + forward.** Two users; the item lands in the author's inbox and can be **forwarded** to a teammate's inbox; the teammate's answer returns to the AI session. (*This completes the canonical demo, §7.2.*)

- **Stage 3 — One rule.** Capture a rule that auto-answers or auto-routes a repeat of the demo question, demonstrating decisions-as-assets at skeleton depth.
- **Stage 4 — Findings & options.** Write up what worked, what was awkward, and a recommendation on the work-item format standard and the rule model — feeding `data-model.md` and the next sprint.

Each stage is a checkpoint: if Stage 0 feels wrong, that's the cheapest possible moment to learn it.

11. Relationship to the existing codebase

This PoC builds **on** the current foundation, not beside it:

- The **ports & adapters** structure (`architecture.md`) already anticipates many faces (web, API, CLI, MCP) over one framework-free core — the MCP tap-in is just another **driving adapter**, and inboxes/work items/rules are new **domain** concepts with their own **ports**.
- The existing **files table** and the planned **R2 storage** are where work-item **attachments** will live.
- The broader relational model (projects → requests → items → files, plus inboxes and rules) still needs designing in `data-model.md` before tables are written — this overview defines the *goals* that design must serve.